

SOFTWARE REQUIREMENTS SPECIFICATION

Milestone 2 — Swipe System & Job Discovery

SwipeX

Swipe-Based Intelligent Job Discovery and Career Assistance Platform

Field	Detail
Document Type	Software Requirements Specification (SRS) — Milestone 2
Milestone Duration	7 Working Days
Team Size	5 Interns, working in parallel across five domains
Team Composition	Frontend & User Experience; Backend & Identity Management; Job & Data Intelligence; AI/ML Career Intelligence; Platform, Testing & Deployment
Builds On	Milestone 1 increment: repository/architecture, Auth + RBAC, finalized DB schema, seeded profile/job data, ATS heuristic stub, local Docker Compose environment
Deployment Scope	Excluded from Milestone 2. Production deployment occurs only after Milestone 4.
Tooling Policy	Every tool used must be free / open-source — no paid subscriptions required anywhere in this milestone
Primary Reference	SwipeX Project Charter (provided PDF), Sections 3–5, and the SwipeX Milestone 1 SRS baseline
Prepared For	Educator / Mentor supervising the intern cohort

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the scope, tasks, responsibilities, dependencies, and integration requirements for Milestone 2 of the SwipeX platform. It builds directly on the Milestone 1 increment — the finalized database schema, working Authentication/RBAC, seeded profile and job data, and the heuristic ATS stub — and translates the project charter's Swipe-Based Job Discovery and Smart Job Search & Filtering modules into a coordinated, day-wise execution plan for five interns working in parallel, so that their individual deliverables converge into a single, functioning system increment by the end of Day 7.

1.2 Scope of Milestone 2

Milestone 2 covers the Swipe System and Job Discovery experience. By the end of Day 7 the team must deliver a working increment consisting of: a live Job Service exposing a searchable, filterable catalog of seeded job listings across MNCs, startups, and newly founded companies; a Swipe Service that records apply/save/skip actions per user; Company & Startup listing pages backed by real data; job freshness and competition-level indicators computed from seeded data; a heuristic Matching Engine that ranks the swipe feed per user using profile fit and swipe history (a rule-based placeholder ahead of the real NLP/ML-driven, ATS-aware ranking scheduled for Milestone 3); a connected frontend with a working swipe deck, filters/search UI, and company pages consuming live backend APIs; and a validated, tested integration of all of the above inside the shared local/dev Docker Compose environment.

Explicitly out of scope for Milestone 2: full NLP/ML-based resume-aware ranking and ATS scoring logic (Milestone 3); application-tracking dashboards, notifications, and analytics (later milestones); and production/cloud deployment, which remains reserved for Milestone 4.

1.3 Definitions & Abbreviations

- SRS — Software Requirements Specification
- RBAC — Role-Based Access Control
- JWT — JSON Web Token, used for stateless authentication
- API Gateway — Single entry point that routes requests to backend services
- Swipe Action — A user's right-swipe (apply/save) or left-swipe (skip) response to a job card, recorded by the Swipe Service
- Matching Engine — The service that ranks jobs for a user using rule-based, heuristic scoring (profile fit + swipe history); a placeholder for the trained AI/ML recommendation model scheduled for Milestone 3
- Competition Level — A Low/Medium/High indicator computed from applicant-count thresholds for a job listing
- M1–M4 — Milestone 1 through Milestone 4, as defined in the project charter
- Intern 1–5 — Shorthand for the five domain owners defined in Section 4

1.4 Reference Documents

- SwipeX: Swipe-Based Intelligent Job Discovery and Career Assistance Platform — Project Charter (source PDF), Section 3 (Architecture Diagram) and Section 4, Modules 3–4 (Swipe-Based Job Discovery; Smart Job Search & Filtering)
- SwipeX Project Charter (source PDF), Section 5 — Milestone 2: Week 3 & 4 (Swipe System & Job Discovery)
- SwipeX Milestone 1 SRS — baseline architecture, database schema, Auth/RBAC, and repository structure this milestone builds on
- Prior mentor/team discussions establishing the five-domain team split and the deployment exclusion through Milestone 4

1.5 Golden Rule for This Milestone

Every tool used in Milestone 2 must be free. No intern should need to pay for anything — free code editors, free version control, free API-testing tools, free local container tooling, free CI minutes, free linting/security tools, and free UI/search libraries are sufficient to complete everything described in this document.

2. Free Tools Checklist

Before Day 1 begins, make sure every intern has access to the free tools below that are relevant to their domain, in addition to the Milestone 1 toolset already in use. No paid tool is required anywhere in this milestone.

What It's Needed For	Free Tool Option(s)
Writing code	VS Code (completely free)
Saving & sharing code	Git + GitHub (free for public/private small repos)
Testing APIs manually	Postman (free plan) or the Thunder Client extension in VS Code (free)
Backend framework	Python + Django REST Framework (free, open-source) or FastAPI (free)
Frontend framework	React.js (free, open-source) with Tailwind CSS (free)
Swipe-gesture UI	react-tinder-card (free, open-source) or a custom Framer Motion drag/gesture implementation (free)
Database	PostgreSQL (free, open-source) for structured data; Redis (free, open-source) for caching
Keyword/full-text search & filtering	PostgreSQL built-in trigram/GIN indexes (pg_trgm, free) — sufficient for Milestone 2 scale; no paid search service required

What It's Needed For	Free Tool Option(s)
Automated backend tests	pytest (Python, free) or Jest (Node.js, free)
Automated local integration tests	Docker Compose (free) running the whole stack on a laptop — no live server needed
CI checks on every PR (lint/build/test only)	GitHub Actions (free tier for public/small private repos) — no deployment stage configured
Code quality / linting	ESLint (frontend, free), Flake8 / Black (backend, free)
API documentation	Swagger UI / OpenAPI (free, built into DRF/FastAPI)
Heuristic Matching Engine environment	Python, pandas, NumPy (all free) — sufficient for rule-based scoring in Milestone 2
Diagrams (architecture, ER diagram)	draw.io / dbdiagram.io (free)
Defect tracking / task board	GitHub Issues & Projects (free) or Trello (free tier)
UI charts / animations (if needed)	Recharts / Chart.js (free, open-source); Framer Motion (free)

None of the tools above require a paid plan to complete Milestone 2. If a paid tier is ever tempting (e.g., a hosted search service), the local/free alternative listed here is sufficient for this milestone's scope.

3. Overall Introduction

Milestone 2 extends the Milestone 1 foundation with the two backend services that make SwipeX feel like SwipeX: the Job Service (job listings, company/startup profiles, search and filtering) and the Swipe Service (recording apply/save/skip actions). Per the project charter's architecture diagram, these sit behind the same API Gateway as the Authentication Service, are backed by the same PostgreSQL/Redis data layer, and are consumed by the React.js frontend's Swipe Interface and Filters & Search components. A first version of the Matching Engine (labelled 'AI Recommendation Service' in the architecture diagram) is also introduced in this milestone as a rule-based, heuristic ranking layer — using profile fit and swipe history rather than a trained model — so the personalized swipe feed can be demonstrated end-to-end ahead of the real NLP/ML-driven recommendation and ATS-aware ranking work scheduled for Milestone 3.

By the end of this milestone, a Job Seeker will be able to log in (Milestone 1), open the swipe deck, see a personalized feed of real seeded jobs, swipe right to apply/save or left to skip, use filters and search to narrow the catalog, and browse company/startup listing pages — all against live backend data running inside the shared Docker Compose environment.

4. Team Structure & Domain Ownership

Intern	Domain	Primary Architecture Ownership (Milestone 2)
Intern 1	Frontend & User Experience	Swipe deck UI, Job Card component, swipe gestures, Filters & Search UI, Company/Startup listing pages, personalized feed hook-up
Intern 2	Backend & Identity Management	Job Service, Swipe Service, Search API, Company API, Gateway extension, RBAC on new endpoints
Intern 3	Job & Data Intelligence	Job/Company/Swipe schema extension, seeded job & company dataset, search indexing, freshness & competition-level computation
Intern 4	AI/ML Career Intelligence	Matching Engine (heuristic recommendation/scoring), swipe-based preference learning v0, Milestone 3 roadmap

Intern	Domain	Primary Architecture Ownership (Milestone 2)
Intern 5	Platform, Testing & Deployment	CI/CD extension for new services, local Docker Compose update, test automation, defect tracking, integration validation

Deployment (packaging for production, cloud release, AWS/Azure hosting) remains excluded from Intern 5's scope in this milestone. It is scheduled after Milestone 4 is complete.

5. Milestone 2 Objectives

- Design and finalize the production-ready Job, Company, SavedJob, and Swipe database schema, extending the Milestone 1 baseline.
- Implement the Job Service: paginated job listings, job detail, and job search/filter APIs (keyword, location, job type, salary range, experience level, skills).
- Implement the Swipe Service: record apply/save/skip actions per user, enforced by RBAC for the Job Seeker role.
- Implement Company & Startup listing and profile APIs backed by seeded data.
- Compute and expose job freshness (posted-time) and competition-level (applicant-count-based) indicators.
- Stand up a heuristic Matching Engine that ranks the swipe feed per user from profile fit and swipe history, ahead of the trained AI/ML model scheduled for Milestone 3.
- Deliver a connected frontend: a working swipe deck, filters/search UI, and company/startup pages consuming live backend APIs, with the swipe feed personalized by the Matching Engine.
- Validate the full increment through integration testing, automated checks, and a defect-tracked QA cycle — without production deployment.

6. Functional Requirements — Milestone 2 Scope

Req ID	Requirement	Owner	Priority
FR-01	Users can browse a paginated list of live job listings and view individual job details	Intern 2	High
FR-02	Users can swipe right (apply/save) or left (skip) on a job card, and the action is recorded	Intern 1 / Intern 2	High
FR-03	System exposes a job search/filter API supporting keyword, location, job type, salary range, experience level, and skills	Intern 2 / Intern 3	High
FR-04	System exposes Company/Startup listing and profile data backed by seeded records	Intern 2 / Intern 3	Medium
FR-05	System computes and exposes job freshness and competition-level indicators per listing	Intern 3	Medium
FR-06	System exposes a heuristic Matching Engine that ranks/recommends jobs per user from profile fit and swipe history	Intern 4	High
FR-07	The frontend swipe deck consumes the personalized recommendation feed instead of a static/chronological list	Intern 1	High
FR-08	The frontend exposes a filters/search UI and Company/Startup listing pages wired to live APIs	Intern 1	High
FR-09	RBAC restricts swipe/apply actions to the Job Seeker role and job/company-management actions to the Recruiter role	Intern 2	High
FR-10	All Milestone 2 services run together in the local Docker Compose integration environment and pass CI checks	Intern 5	High

7. Cross-Team Dependency Matrix

This matrix makes explicit which artifact each domain must hand off, to whom, and by which day, so that no intern is idle or blocked waiting on an undefined interface.

Consumer	Depends On	Artifact Needed	Needed By
Intern 2	Intern 3	Extended Job/Company/Swipe schema + seed dataset	Day 2
Intern 4	Intern 3	Seeded job dataset + schema fields for scoring	Day 1–2
Intern 1	Intern 2	Live GET /jobs and POST /swipes APIs	Day 3
Intern 4	Intern 2	Live Swipe API (interaction history)	Day 3
Intern 1	Intern 2 / Intern 3	Live Search API + Company API + competition/freshness fields	Day 4
Intern 1	Intern 4	Recommendation (Matching Engine) API	Day 5
Intern 2	Intern 4	Matching Engine routes to register in the API Gateway	Day 4–5
Intern 5	Intern 1 / 2 / 3 / 4	Dev-deployed endpoints/UI for each service as they land	Days 2–5

8. Master 7-Day Execution Timeline

All five interns work in parallel every day, each building directly on the previous day's output so no one is left waiting idle. The table below is the single-glance view; Section 9 gives the full detail (task/output/dependency/hand-off) per intern.

Day	Intern 1 Frontend	Intern 2 Backend/Identity	Intern 3 Job & Data	Intern 4 AI/ML	Intern 5 QA/DevOps
1	Wireframes + swipe-library evaluation	Job/Swipe API contracts + Gateway plan	Job/Company/Swipe schema extension + seed dataset design	Matching Engine feature set + scoring v0 design	CI/docker-compose extension + M2 test plan
2	Job Card + swipe gesture (mock data)	GET /jobs, GET /jobs/{id}, POST /swipes live	Seed data loaded + filter indexes built	Matching Engine skeleton + stub /recommendations	Test cases + automated suite scaffold
3	Swipe UI wired to live Job/Swipe APIs	/jobs/search + /companies APIs + RBAC	Company schema/seed + query optimization	Matching Engine uses live swipe history	Functional tests on Job/Swipe/Search; CI runs suite
4	Filters/Search UI + Company pages live	Freshness/competition fields + Gateway route for Matching Engine	Competition-level & applicant-count computation	Tune scoring + reason tags + unit tests	Integration round 1 across all services
5	Personalized feed hook-up + RBAC views	Full Gateway integration of all M2 services	Support integration + backup/restore update	Finalize Matching Engine integration + M3 roadmap	Full M2 integration test cycle
6	UI polish, responsiveness, defect fixes	End-to-end backend integration + Swagger docs	Unit tests + query optimization + defect fixes	Unit tests for scoring + defect fixes	Regression testing + code-quality report
7	Final UI fixes + demo walkthrough prep	Code freeze + sign-off summary	Final validation + M3 handoff notes	Final review + M3 handoff doc	Final QA sign-off report + M2 test summary

9. Detailed Intern-Wise Task Breakdown

Each table below lists, for every day, the task, the concrete expected output, what the intern depends on to start that task, and what they hand off to the rest of the team. Every day's task is designed to produce a usable artifact for at least one other intern, so no one is left without work while waiting on a dependency.

9.1 Intern 1 — Frontend & User Experience

Day	Task Description	Expected Output / Deliverable	Depends On	Provides To
1	Review Intern 2's draft Job/Swipe API contracts and Intern 3's schema notes; produce wireframes for the swipe deck, job card, filters panel, and company/startup listing pages; evaluate and select a swipe-gesture library (e.g., react-tinder-card) or plan a custom Framer Motion implementation.	Wireframes for swipe deck, filters, and company pages; swipe-library decision documented	Intern 2's draft API contracts; Intern 3's schema notes	Self (Day 2 build)
2	Build the reusable Job Card component (logo, title, company, salary, location, tags) and implement swipe gesture interactions (right = apply/save, left = skip) against mocked data; scaffold the Filters panel UI (MNC/Startup/Remote/Salary/Experience controls).	Job Card + swipe gesture working on mock data; Filters panel scaffolded	Own wireframes	Self (Day 3 live integration)
3	Wire the swipe deck to Intern 2's live GET /jobs API and POST /swipes API; implement optimistic UI updates and an undo-last-swipe control.	Swipe deck functioning against live Job/Swipe APIs	Intern 2's Job listing & Swipe APIs	Intern 5 (swipe-flow test target)
4	Wire the Filters & Search UI to Intern 2's live /jobs/search API; build Company/Startup listing pages consuming /companies; add Advanced Filter chips (fresher-friendly, low competition, recently posted).	Filters/Search and Company pages functioning against live APIs	Intern 2's Search & Company APIs; Intern 3's competition/freshness fields	Intern 5 (filters/company test target)
5	Replace the chronological swipe queue with Intern 4's personalized recommendation feed; implement RBAC-aware views (Job Seeker swipe deck vs Recruiter job-posting form); polish loading/error states for filters and search.	Personalized, RBAC-aware swipe experience	Intern 4's Recommendation API; Intern 2's RBAC claims	Intern 5 (full UI test target)
6	UI polish across swipe deck, filters, and company pages; responsiveness fixes; empty-state and error handling; fix defects reported by QA.	Stable, tested frontend build	Intern 5's defect log	Intern 5 (regression build)
7	Final UI fixes from QA feedback; rehearse the Milestone 2 demo walkthrough (swipe → filter → company page → personalized feed); document frontend module notes.	Demo-ready frontend + module documentation	Passed regression	Milestone 2 demo

9.2 Intern 2 — Backend & Identity Management

Day	Task Description	Expected Output / Deliverable	Depends On	Provides To
1	Draft the OpenAPI contract for the Job Service and Swipe Service (GET /jobs, GET /jobs/{id}, POST /swipes, GET /jobs/search, GET /companies); extend the API Gateway routing plan for the new services; review the Job/Company/Swipe schema draft with Intern 3.	Draft OpenAPI contracts for Job/Swipe/Search/Company endpoints; Gateway routing plan updated	Intern 3's schema draft	Intern 1 & Intern 4 (contracts to build against)
2	Implement GET /api/v1/jobs (paginated list) and GET /api/v1/jobs/{id} backed by Intern 3's seed data; implement POST /api/v1/swipes (records apply/save/skip) with RBAC restricting the action to the Job Seeker role; register the new routes in the API Gateway.	Working Job listing & Swipe endpoints, RBAC-protected and registered	Intern 3's seeded, migrated schema; Intern 5's updated dev environment	Intern 1 (live data to build against), Intern 4 (swipe history source), Intern 5 (test target)
3	Implement GET /api/v1/jobs/search (keyword, location, job type, salary range, experience level, skills) and GET /api/v1/companies, GET /api/v1/companies/{id}; enforce RBAC so Recruiters can manage company/job data while Job Seekers can only browse.	Working Search and Company endpoints, RBAC enforced	Intern 3's indexed schema and company/startup seed data	Intern 1 (filters & company pages), Intern 5 (test target)
4	Expose job freshness and competition-level fields (posted_time, applicant_count, competition_level) computed by Intern 3 in job responses; register Intern 4's Matching Engine route behind the Gateway; add a minimal Recruiter job-posting CRUD stub to seed realistic flow.	Freshness/competition data exposed via API; Matching Engine reachable through the Gateway; Recruiter job-posting stub live	Intern 3's freshness/competition computation; Intern 4's Matching Engine skeleton	Intern 1 (advanced filters & personalized feed), Intern 5 (integration round 1)
5	Integrate all Milestone-2 services (Job, Swipe, Search, Company, Matching Engine) fully behind the API Gateway; update the shared Postman collection; pair with Intern 5 on the full integration session.	Unified backend for Milestone 2 behind one Gateway	Intern 3 & Intern 4's completed endpoints	Intern 1 & Intern 5 (single integrated backend to consume/test)
6	Run an end-to-end backend integration pass; triage and fix defects raised by QA; finalize Swagger/OpenAPI documentation for all Milestone 2 endpoints.	Stable integrated backend; published API docs	Intern 5's defect log	Intern 5 (fixed build for regression)
7	Code freeze for the Milestone 2 backend; support the final demo; write a backend architecture & integration summary for sign-off.	Backend Milestone 2 sign-off report	Passed QA regression	Milestone 2 demo / mentor review

9.3 Intern 3 — Job & Data Intelligence

Day	Task Description	Expected Output / Deliverable	Depends On	Provides To
1	Extend the Milestone 1 schema into production-ready Job, Company, SavedJob, and Swipe tables; design a realistic seed dataset (100+ listings) spanning MNCs, startups, and newly founded companies with salary, location, skills, experience, and posted_at fields; plan search indexing (PostgreSQL trigram/GIN).	Extended schema design + seed dataset plan; indexing strategy documented	Intern 2's Milestone 1 schema baseline	Intern 2 (schema to build APIs against), Intern 4 (fields for scoring)
2	Run migrations for the extended schema; load the seed dataset into the dev database; build indexes on filterable fields (location, job_type, salary_range, skills).	Migrated, seeded database with filter-ready indexes	Own schema design; Intern 5's DB container	Intern 2 (data to serve via APIs), Intern 4 (data for scoring)
3	Add the Company/Startup profile table with seed data (name, about, size, industry, 'newly founded' flag); optimize DB queries for the search/filter combinations Intern 2 flags.	Company/startup data live; optimized query layer	Intern 2's search/filter usage patterns	Intern 2 (company data to serve), Intern 1 (company page content)
4	Implement rule-based computation of job freshness (posted_time-based) and competition level (applicant-count thresholds → Low/Medium/High); migrate these fields into the Job table; validate index performance under combined filter+search load.	Freshness/competition fields live and performant	Own schema; Intern 2's API usage	Intern 2 (fields to expose), Intern 1 (advanced filters)
5	Support the full cross-team integration session; validate data integrity under concurrent swipe writes; update the DB backup/restore script to cover the new Milestone 2 tables.	Verified data layer under integration load; updated backup/restore script	Intern 2's gateway integration	Intern 5 (stable data layer for integration tests)
6	Write unit tests for the repository/query layer (job listing, search, swipe history); optimize any slow queries flagged during integration testing; fix defects from QA.	Tested, optimized data service	Intern 5's defect log	Intern 5 (regression build)
7	Final validation of the data module; finalize schema documentation and data dictionary updates; write handoff notes for Milestone 3 (what the Resume/ATS Engine will need from Job data).	Module sign-off + Milestone 3 handoff notes	Passed regression	Milestone 2 demo

9.4 Intern 4 — AI/ML Career Intelligence

Day	Task Description	Expected Output / Deliverable	Depends On	Provides To
1	Define the feature set for the Matching Engine (skills overlap, location match, experience fit, swipe-history signals); design a rule-based weighted scoring formula v0; review with Intern 3 which Job schema fields are needed.	Feature list + scoring formula v0 documented	Intern 3's schema draft	Self (Day 2 build)
2	Scaffold the Matching Engine microservice (FastAPI) with a stub GET /api/v1/recommendations?user_id= endpoint returning a heuristic-ranked job list computed against the seed dataset.	Matching Engine skeleton with a working stub recommendation endpoint	Intern 3's seeded dataset	Intern 2 (route to register), Intern 1 (contract to build against)
3	Integrate the Matching Engine with Intern 2's live Swipe API so subsequent recommendations adjust based on like/skip history; finalize the response schema (job_id, match_score, reason_tags).	Matching Engine responding to live swipe history	Intern 2's Swipe API	Intern 1 (personalized feed data), Intern 2 (integration)
4	Tune scoring weights using the seeded swipe interaction data; add human-readable 'reason tags' (e.g., 'Matches your skills', 'Low competition'); write unit tests for the scoring function.	Tuned scoring with reason tags; unit tests passing	Intern 3's competition/freshness fields	Intern 1 (richer recommendation display)
5	Finalize Matching Engine integration behind the API Gateway with Intern 2; document the scoring methodology and a Milestone 3 roadmap note describing how this heuristic evolves into full AI/ML-driven, ATS-aware ranking.	Matching Engine fully integrated; M3 roadmap doc	Intern 2's gateway	Intern 1 (display contract), Mentor (M3 plan)
6	Write additional unit tests for the scoring/reason-tag logic; fix defects surfaced during QA's integration cycle.	Tested Matching Engine	Intern 5's defect log	Intern 5 (regression build)
7	Final review of the Matching Engine module; document the Milestone 2 scope boundary (heuristic personalization only) and the handoff plan for full AI/ML recommendation and ATS-aware ranking in Milestone 3.	Module sign-off + Milestone 3 handoff document	Passed regression	Milestone 2 demo

9.5 Intern 5 — Platform, Testing & Deployment

Scope note: Intern 5's Milestone 2 work remains limited to QA, integration validation, code-quality tooling, test automation, and defect tracking, plus updates to the local Docker Compose environment. No production packaging or deployment activity is performed in Milestone 2; that work resumes only after Milestone 4.

Day	Task Description	Expected Output / Deliverable	Depends On	Provides To
1	Extend CI to cover the new Job/Swipe/Company/Matching-Engine service folders; extend docker-compose.dev.yml to add the new services; update the Milestone 2 test plan and defect-tracking board.	CI/docker-compose updated for Milestone 2 scope; test plan & defect board ready	Intern 2's service folder structure	All interns (updated shared dev environment)
2	Write test cases for Job listing and Swipe endpoints; scaffold an automated test suite (pytest/Postman) for the new services; confirm CI runs build/lint on the new service code.	Test case document; automated suite scaffolded for Job/Swipe	Intern 2's draft API contracts	Self (Day 3 execution)
3	Execute functional tests against the live Job, Swipe, and Search APIs as they land; log defects with severity/priority; extend CI to run the new automated suite on every PR.	Day-3 test execution report; logged defects; CI running the new suite	Intern 2's live endpoints	Intern 2 & 3 (defect list to fix)
4	Run integration round 1 across Frontend + Job/Swipe/Search/Company + Matching Engine inside docker-compose.dev.yml; log and prioritize defects.	Integration round 1 report; prioritized defect list	All four other interns' Day-4 endpoints/UI	All interns (defect backlog for Day 5)
5	Run the first full Milestone 2 integration test cycle end-to-end (Job discovery + Swipe + Search/Filter + Recommendation feed + Company pages); verify RBAC behavior; log and prioritize defects for Day 6.	Full integration test report; prioritized defect list	All four other interns' Day-5 integrated build	All interns (defect backlog for Day 6)
6	Run regression tests after fixes; check automated test coverage; run static code-quality analysis; support interns closing out integration bugs.	Regression report; code-quality report	Fixes from Interns 1–4	All interns (go/no-go signal)
7	Run the final Milestone 2 validation cycle (smoke + integration + RBAC + UI checks); compile the QA sign-off report and test summary for the demo. No deployment activity is performed.	Milestone 2 QA sign-off report + test summary	Clean regression build	Mentor / Milestone 2 review

10. Day-Wise Work Plan (Plain Language + Tick-Box Checklist)

Every day starts with a 15-minute stand-up. Tick each checkpoint box only when that item is actually working, not just attempted. No deployment tasks appear anywhere in this plan — everything runs locally via Docker Compose or against the shared dev environment.

10.1 Intern 1 — Frontend & User Experience

Owens: Swipe Deck UI, Filters & Search, Company/Startup Pages

Day	What To Do (Explained Simply)	Checkpoints — Tick When Done
1	Look at the plans for the Job and Swipe APIs, and sketch out simple designs for the swipe cards, filters, and company pages — like designing the 'look and feel' before building. Also pick a tool that makes cards swipeable.	- Wireframes made for swipe deck, filters, and company pages - Swipe-gesture library chosen (or custom plan made) - Plan shared with the team
2	Build the actual job card (with photo/logo, title, salary) and make it swipeable left/right using fake/mock data; start building the filters panel.	- Job Card built and swipeable (mock data) - Filters panel scaffolded - Ready to connect to real APIs
3	Connect the swipe deck to Intern 2's real job/swipe APIs, so swiping right/left actually does something; add an 'undo my last swipe' button.	- Swipe deck connected to live Job & Swipe APIs - Undo-last-swipe working - Tested swiping right and left
4	Connect the filters and search bar to the real search API, and build pages where users can browse companies and startups.	- Filters/Search connected to live API - Company/Startup pages built and showing real data - Advanced filter chips added (fresher-friendly, low competition, recently posted)
5	Replace the plain job list with Intern 4's personalized recommendation feed, so users see jobs picked for them; make sure Job Seekers and Recruiters see different screens.	- Personalized feed connected and working - Role-based views working (Job Seeker vs Recruiter) - Loading/error messages added
6	Polish the app — fix rough edges, make sure it works well on phones/tablets, and show friendly messages if something breaks.	- Responsive design checked - Friendly error/empty states added - Bugs reported by QA fixed
7	Final Integration Day — fix any last issues and rehearse a simple walkthrough of swipe → filter → company page → personalized feed.	- Final test with full team done - Demo walkthrough rehearsed - Frontend notes documented

10.2 Intern 2 — Backend & Identity Management

Owns: Job Service, Swipe Service, Search & Company APIs, Gateway Extension

Day	What To Do (Explained Simply)	Checkpoints — Tick When Done
1	Sketch out, in simple terms, what the Job and Swipe APIs will look like, and plan how the Gateway will route to these new services. Check in with Intern 3 about the data shape.	- Draft API contract for Job/Swipe/Search/Company written - Gateway routing plan updated - Reviewed schema plan with Intern 3
2	Build the actual API that lists jobs and shows job details, and the API that records when someone swipes right or left. Make sure only Job Seekers can swipe.	- Job listing & job detail API working - Swipe API working and recording actions - Role rule added (only Job Seekers can swipe)
3	Build the search/filter API (so people can search by keyword, location, salary, etc.) and the API that shows company/startup info. Make sure Recruiters can manage jobs but Job Seekers can only browse.	- Search/filter API working - Company API working - Role rules tested (Recruiter vs Job Seeker)
4	Show how 'new' or 'competitive' a job is on the API, and connect Intern 4's recommendation service to the same Gateway so it's reachable. Add a simple way for Recruiters to post a job.	- Freshness/competition info showing in job data - Matching Engine reachable through the Gateway - Basic 'post a job' option for Recruiters
5	Connect everything (jobs, swipes, search, companies, recommendations) behind one single Gateway, and update the shared testing collection.	- All Milestone 2 services reachable through one Gateway - Shared Postman collection updated - Joint test done with Intern 5
6	Run the whole backend together, fix anything QA finds broken, and write simple documentation of what each API does.	- End-to-end backend test done - Bugs reported by QA fixed - API documentation updated
7	Final Integration Day — freeze the backend code, help demo everything, and write a short summary of what was built.	- Backend code frozen - Took part in full team demo - Short summary document written

10.3 Intern 3 — Job & Data Intelligence

Owns: Job/Company/Swipe Schema, Seed Data, Search Indexing

Day	What To Do (Explained Simply)	Checkpoints — Tick When Done
1	Plan out exactly how job, company, and swipe information will be stored, and design a big sample list of realistic jobs (from big companies, startups, and new companies).	• Job/Company/Swipe table design finalized • Sample job dataset planned (100+ listings) • Search strategy planned
2	Actually create those database tables and load in the sample job data; build 'shortcuts' (indexes) so filtering by location/salary/skills is fast.	• Tables created and migrated • Sample job data loaded • Filter indexes built
3	Add company/startup profile information (name, about, size, industry) with sample data, and speed up any slow searches Intern 2 points out.	• Company/startup data loaded • Slow queries optimized • Data shared with Intern 2
4	Add simple rules to mark a job as 'new' (recently posted) or show its 'competition level' (low/medium/high based on how many people applied).	• Freshness rule working • Competition-level rule working • Performance checked under filters+search together
5	Join the team's full test session, make sure the data stays correct even when lots of swipes happen at once, and update the backup script to cover the new tables.	• Data verified correct under test load • Backup/restore script updated • Joint test done with Intern 2
6	Write simple automated tests for your job/search data, and fix any slow queries or bugs QA finds.	• Automated tests written • Slow queries fixed • Bugs reported by QA fixed
7	Final Integration Day — confirm everything works with the rest of the team, and write handoff notes for what Milestone 3 (the AI Resume Engine) will need from this data.	• Final test with full team done • Schema documentation finalized • Handover notes for Milestone 3 written

10.4 Intern 4 — AI/ML Career Intelligence

Owens: Matching Engine (Heuristic Recommendation)

Day	What To Do (Explained Simply)	Checkpoints — Tick When Done
1	Think through what makes a job a 'good match' for someone (matching skills, location, experience) and write a simple scoring formula plan. Check with Intern 3 what data fields you'll have to work with.	- List of matching factors written down - Simple scoring formula v0 planned - Reviewed available data fields with Intern 3
2	Build a very simple 'Matching Engine' service with one endpoint that returns a ranked list of jobs using your scoring formula on the sample data.	- Matching Engine service created /recommendations endpoint working - Returns a ranked sample list
3	Connect your Matching Engine to real swipe data, so recommendations start changing based on what a person likes/skips.	- Connected to live swipe history - Recommendations respond to likes/skips - Response format finalized
4	Fine-tune your scoring so it feels smarter, and add short reasons next to each job (like 'Matches your skills' or 'Low competition').	- Scoring tuned using real swipe data - Reason tags added - Tests written for the scoring logic
5	Connect your service to the shared Gateway with Intern 2, and write a short plan for how this will become a real AI model in Milestone 3.	- Matching Engine integrated behind the Gateway - Milestone 3 roadmap document written - Joint test done with Intern 1
6	Write more tests for your scoring logic, and fix anything QA finds broken.	- Additional tests written - Bugs reported by QA fixed - Service confirmed stable
7	Final Integration Day — join the team test, and clearly write down that this is still simple rule-based matching, not a trained AI model yet.	- Final test with full team done - Scope clearly documented (heuristic only — real AI model comes in Milestone 3) - Handover notes written

10.5 Intern 5 — Platform, Testing & Deployment

Owens: CI/CD Extension, Test Automation, Integration Validation (No Deployment)

Day	What To Do (Explained Simply)	Checkpoints — Tick When Done
1	Update the automatic checks (CI) and the shared local environment (Docker Compose) so they now include the new job/swipe/matching services; update the test plan and bug tracker for this milestone.	- CI updated for new services - docker-compose updated for new services - Test plan & bug tracker updated
2	Write down what needs testing for the job listing and swipe features, and set up a tool to test those automatically.	- Test cases written for Job & Swipe APIs - Automated test tool set up - CI running lint/build on new code
3	Actually run tests against the real job/search APIs as they go live, and note down anything broken.	- Job/Swipe/Search APIs tested - Bugs logged with priority - CI now running the new automated tests
4	Bring the frontend and all the new backend services together and test the whole flow once, noting down anything that doesn't work.	- First full integration test run done - Issues logged and prioritized - Shared with the team for Day 5 fixes
5	Run one big, full test of the whole journey — swiping, filtering, seeing companies, and getting recommendations — and check that role permissions work correctly.	- Full journey tested end-to-end - Role-based access tested - All issues logged for Day 6
6	Re-test everything after the team's fixes, and run a quality check on the code to catch anything messy.	- Regression testing done - Code-quality report generated - System confirmed stable
7	Final Integration Day — run one last full check, write a summary of what was tested, and confirm no deployment work was done.	- Final full test cycle completed - QA sign-off report written - Confirmed: no deployment activity performed this milestone

11. Daily Coordination & Integration Sync Protocol

11.1 Daily Standup (15 minutes, start of day)

- Each intern states: what was completed yesterday, what is planned today, and any blocker relative to the Dependency Matrix (Section 7).
- Intern 2 (Backend & Identity Management) tracks blockers and re-sequences work if a dependency is late.

11.2 Integration Checkpoints

Checkpoint	Day	Purpose
Contract Freeze	Day 2 (EOD)	Job, Swipe, Search, and Company API contracts are frozen; no breaking changes without team sign-off
First Vertical Slice	Day 3–4	Swipe deck + Filters/Search reachable end-to-end from the frontend through the Gateway
Full Integration	Day 5	All Milestone 2 services (Job, Swipe, Search, Company, Matching Engine, Frontend) run together in docker-compose.dev.yml and pass smoke tests
Regression & Sign-off	Day 7	Final validation cycle passes; Milestone 2 increment is signed off by QA and the mentor

11.3 Change Control

Any change to a frozen interface contract after Day 2 must be raised by the owning intern in standup, agreed by all consumers listed in Section 7, and reflected the same day in the shared Postman/pytest collection maintained by Intern 5.

12. Milestone 2 Deliverables Checklist

Deliverable	Owner	Due
Extended Job/Company/Swipe schema + seed dataset	Intern 3	Day 2
Frozen API contracts (Job, Swipe, Search, Company)	Intern 2 / 3	Day 2
Working Job Service + Swipe Service (RBAC enforced)	Intern 2	Day 2–3
Working Search API + Company/Startup API	Intern 2 / 3	Day 3–4
Job freshness & competition-level indicators	Intern 3	Day 4
Heuristic Matching Engine (recommendation feed)	Intern 4	Day 2–5
Swipe deck UI + Filters/Search UI + Company pages (frontend)	Intern 1	Day 2–5
Personalized, RBAC-aware swipe experience	Intern 1 / Intern 4	Day 5
CI/docker-compose updated for Milestone 2 services	Intern 5	Day 1–2
Test plan, test cases, defect log	Intern 5	Day 1–7 (continuous)
Full integration inside local Docker Compose	All (led by Intern 2 & 5)	Day 5
Regression report + code-quality report	Intern 5	Day 6
Milestone 2 sign-off report + demo	All	Day 7

13. Milestone 2 Acceptance Criteria

- The Job Service exposes a paginated, searchable, filterable catalog of live seeded job listings across MNCs, startups, and newly founded companies.
- The Swipe Service correctly records apply/save/skip actions per user, restricted to the Job Seeker role via RBAC.
- Company & Startup listing and profile pages are backed by live seeded data.
- Job freshness and competition-level indicators are computed correctly and exposed on job listings.
- The heuristic Matching Engine ranks jobs per user from profile fit and swipe history and is reachable end-to-end through the Gateway.
- The frontend swipe deck consumes the personalized recommendation feed instead of a static/chronological list.
- The Filters/Search UI and Company/Startup pages are wired to live backend APIs.
- All Milestone 2 services run together and pass integration/smoke tests inside the local Docker Compose environment.
- CI enforces lint, build, and automated test checks on every pull request.
- Every tool used across the milestone was free/open-source — no paid subscriptions were required.
- No production deployment activity has occurred; deployment remains explicitly scheduled for after Milestone 4.

14. Risks & Mitigations

Risk	Impact	Mitigation
Interface contracts (Job/Swipe/Search/Company) change after Day 2, breaking downstream work	High	Contract freeze checkpoint (Section 11.2) + change-control process (Section 11.3)
Seed job/company dataset too small or unrealistic, limiting filter/search/matching demo value	Medium	Intern 3 targets 100+ realistic listings across all company types by Day 2; Intern 4 flags gaps early
Matching Engine scoring feels arbitrary without real user behavior data	Medium	Intern 4 tunes weights against seeded swipe interactions from Day 3 onward rather than only at the end
Swipe-gesture library integration issues (mobile/touch vs desktop/mouse)	Medium	Intern 1 evaluates the library on Day 1 before committing; falls back to a custom Framer Motion implementation if needed
Local integration environment differs from teammates' machines	Medium	Single shared docker-compose.dev.yml maintained by Intern 5; all integration testing happens inside it
Late defect discovery on Day 6–7 with no buffer	Medium	Continuous QA involvement from Day 3 onward, with two full integration rounds on Day 4 and Day 5, rather than testing only at the end
Team accidentally starts deployment work	Low	Reminder in every stand-up that deployment is out of scope for Milestone 2 and reserved for Milestone 4

15. Non-Functional Requirements — Milestone 2 Scope

NFR ID	Requirement	Category
NFR-01	Job listing and search APIs respond within acceptable dev-environment latency for the seeded dataset (target: under 500ms for typical filter/search queries)	Performance
NFR-02	Swipe actions are recorded reliably even under rapid, repeated swiping (no dropped or duplicated writes)	Reliability

NFR ID	Requirement	Category
NFR-03	RBAC is enforced consistently across all new Job/Swipe/Company endpoints; unauthorized roles receive a clear 403 response	Security
NFR-04	The swipe deck UI remains responsive and usable on both desktop and mobile-width viewports	Usability
NFR-05	Filter and search indexes scale to at least the Milestone 2 seed volume (100+ listings) without noticeable query degradation	Scalability
NFR-06	All Milestone 2 services and their dependencies run entirely within the shared docker-compose.dev.yml with no external paid services	Portability / Cost

16. Sign-Off

Role	Name	Signature / Approval Date
Educator / Mentor		
Intern 1 — Frontend & User Experience		
Intern 2 — Backend & Identity Management		
Intern 3 — Job & Data Intelligence		
Intern 4 — AI/ML Career Intelligence		
Intern 5 — Platform, Testing & Deployment		